

Print additional (leading and trailing) context lines before and after the match (*grep -C <NUM>*):

```
[manish@clone ~]$ grep -C2 'carry' testfile
this

is test file

to carry out few regular expressions

practical with grep

123 456
```

As you can see, this has printed out two lines before and after the single match found in the file; if there are multiple matches, *Grep* inserts a line containing '-' between each group of lines (each match and its context lines).

Print the filename for each match (*grep -H <pattern> filename*):

```
[manish@clone ~]$ grep -H 'a' testfile
testfile: to carry out few regular expressions

testfile: practical with grep
```

Now, let's run the search a bit differently:

```
[manish@clone ~]$ cat testfile | grep -H 'a'

(standard input): to carry out few regular expressions

(standard input): practical with grep
```

When the stream that *Grep* is asked to search is passed to its standard input via a pipe from a previous command in the chain, *Grep -H* displays (*standard input*) as the filename.

Run in 'quiet' mode (*Grep -q*): When run with this flag, *Grep* does not write anything to standard output, but sets its return value (also known as exit status) to reflect whether a match was found or not. This option is mainly used in scripts that need to check if a given file contains a particular match. A return status of 0 (zero) indicates that a match was found; 1 indicates that no match was found.

```
[manish@clone ~]$ grep -q '2010' testfile
[manish@clone ~]$ echo $?
1
[manish@clone ~]$ grep -q '456' testfile
[manish@clone ~]$ echo $?
0
```

Using regular expressions

```
[manish@clone ~]$ grep 'c.r' testfile
to carry out few regular expressions
```

In the search above, '.' is used to match any single character—which is why it matches 'car' in 'carry'. *Grep* has a powerful regular expression matching engine, which we can't hope to cover in depth here, but we will include a few important points:

- Most characters, including all letters and digits, are actually regular expressions that match themselves.
- Any meta-character (with special meaning to *Grep*, like the '.' in the example above) may be quoted by preceding it with a backslash. This makes *Grep* treat it as an ordinary character.

```
[manish@clone ~]$ grep 'c\.r' testfile
[manish@clone ~]$
```

As you can see, preceding the '.' with a backslash has removed its significance as a meta-character.

A regular expression may be followed by one of several repetition operators:

- The period '.' matches any single character.
- '?' means that the preceding item is optional, and if found, will be matched at the most, once.
- '*' means that the preceding item will be matched zero or more times.
- '+' means the preceding item will be matched one or more times.
- '{n}' means the preceding item is matched exactly n times, while '{n,m}' means the item is matched n or more times. '{n,m}' means that the preceding item is matched at least n times, but not more than m times. '{m}' means that the preceding item is matched, at the most, m times.

However, the repetition operators are part of GNU *Grep*'s extended regular expression syntax, so to use these effectively, remember to add the *-E* option to your command.

For an introduction to more of *Grep* regular expression features, see www.panix.com/~elflord/unix/Grep.html#expressions. For more information on regular expression syntax, refer to the *Regular Expressions* chapter in the *Grep* manual, at www.gnu.org/software/Grep/manual/html_node/Regular-Expressions.html. Meanwhile, we will present some examples of regular expressions and try to show how they work.

Character classes in regular expressions

The 'character class' tool is one of the more flexible and often-used features of regular expressions.

There are two basic ways to use character classes: to specify a list of characters (for example, '[aeiou]' is a list of vowel characters), or a range (like '[m-l]', which expands to '[mnpqrst]'). Ranges are a convenience that saves having to type an entire sequence of characters. A character class can also include a list of special characters, but they can't be used as a range.

A single character class instance will match only one character; to match multiple occurrences of the class, you would need to add a repetition operator, like those mentioned above. For example, to find an eleven-letter string comprising only lower-case alphabets, the regex would be: '[a-z]{11}'. As mentioned earlier, to use the repetition operators, we need to